

Lectures on Monte Carlo Theory

Chapter 2: The Theory of Generators — Part 1 (Sections 2.1–2.3)

Based on Leskeä & Vihola

Chapter 2 Outline (First Half)

- 1 2.1 Introduction
- 2 2.2 Random Permutations
- 3 2.3 Pseudorandom Number Generators in Python

2.1 Introduction: Why Do We Need Random Numbers on a Computer? I

Randomness is the cornerstone of all Monte Carlo methods. Every algorithm in this course requires a reliable source of *independent, identically distributed uniform random numbers*: numbers drawn from the interval $[0, 1)$ such that each draw is completely unrelated to all the others. We need such numbers to simulate stochastic processes, to estimate difficult integrals by sampling, to generate cryptographic keys, and to perform statistical resampling (bootstrapping).

A digital computer, however, is a *deterministic machine*: given the same input and the same program, it always produces the same output, step for step. A computer therefore cannot produce *genuinely* random numbers — there is no physical source of unpredictability. Instead, a computer runs a carefully designed deterministic algorithm that produces a numerical sequence whose *statistical properties* are indistinguishable, in practice, from those of a truly random sequence. Such an algorithm is called a **pseudorandom number generator (PRNG)**. The word “pseudo” comes from the Greek *ψευδης* (pseudeis), meaning “false”; a PRNG’s numbers are not truly random, but they are designed to look random to any practical test.

A motivating failure: the Middle-Square Method (von Neumann, 1940s). The idea seems deceptively clever: start with a four-digit integer (called the *seed*, the starting value), square it, keep the four middle digits as the next number, and repeat. For example, starting from the seed 43:

2.1 Introduction: Why Do We Need Random Numbers on a Computer? II

$$43^2 = 184\mathbf{9}, \quad 49^2 = 24\mathbf{01}, \quad 01^2 = \mathbf{0001}, \quad 00^2 = 0000 \rightarrow 0000 \rightarrow 0000 \rightarrow \dots$$

The sequence collapses to the constant value 0000 after just four steps and stays there forever. This illustrates the fundamental problem: any deterministic algorithm operating on a *finite* collection of possible states must eventually revisit a state it has already been in, and from that point it repeats the same outputs in the same cycle forever. Without very careful design, this repeating cycle can be embarrassingly short, making the generator useless for any application needing more than a handful of numbers.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? III

Definition 2.1.1 — Pseudorandom Number Generator (PRNG)

A PRNG is described by a 5-tuple (S, s_0, f, V, g) where each component has a precise role:

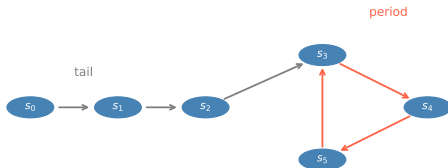
- S is the **state space**: the finite set of all possible internal configurations (memory states) of the generator. The size of S , written $|S|$, is the total number of states.
- $s_0 \in S$ is the **initial state**, also called the *seed*. Changing the seed produces a completely different output sequence, which is essential for reproducibility: recording the seed allows you to reproduce any simulation exactly.
- $f : S \rightarrow S$ is the **transition function**: it maps each state to the next state via $s_{n+1} = f(s_n)$. This is the “engine” of the generator, applied once per output value.
- V is the finite set of **output values**, for example all integers in $\{0, 1, \dots, 2^{32} - 1\}$.
- $g : S \rightarrow V$ is the **output function**: the i -th output value is $u_i = g(s_i)$, for $i = 0, 1, 2, \dots$

Because the state space S is finite, the sequence of states $(s_0, s_1, s_2, \dots)$ must eventually revisit a state it has already visited. Once it does, the generator repeats the same outputs in the same order from that point forward. The length of this repeating cycle is called the **period** of the generator. The period can

2.1 Introduction: Why Do We Need Random Numbers on a Computer? IV

be at most $|S|$ — if every state is visited before any is repeated, the period equals $|S|$. Longer periods are better because the sequence takes more steps before repeating, reducing the risk that a single simulation inadvertently uses the same “random” values twice.

PRNG state sequence: tail + periodic cycle



2.1 Introduction: Why Do We Need Random Numbers on a Computer? V

The figure above illustrates the typical life-cycle of a PRNG state sequence: there is a “tail” of distinct states leading into a cycle (shown as a loop), after which the generator repeats indefinitely. A well-designed PRNG has a period close to $|S|$ and the tail as short as possible.

Key Insight: Period is Everything

For any simulation requiring n samples, the period must be much larger than n . If the period is smaller than n , the sequence simply starts repeating within the simulation, producing spurious correlations that can silently corrupt every result. For modern simulations with $n \approx 10^9$ samples, a period of 2^{128} (as in PCG64, the generator recommended throughout this course) is essentially inexhaustible: even at 10^9 samples per second, one would need 10^{19} times the age of the universe to cycle through the entire period once.

2.1 Introduction: Why Do We Need Random Numbers on a Computer? VI

Definition 2.1.1.1 — Three Equivalent Types of Random Sequences

Three representations of “random numbers” appear throughout the literature. They are interconvertible, and a good PRNG must simultaneously approximate all three.

(i) Uniform numbers from $[0, 1)$. We say $U_1, U_2, \dots$ is a sequence of random numbers from the unit interval $[0, 1)$ if each U_j is a random variable with the *uniform distribution* on $[0, 1)$. This means the probability that U_j falls in any sub-interval $[a, b) \subset [0, 1)$ is exactly the length $b - a$ of that interval:

$$\mathbb{P}(U_j \leq t) = \begin{cases} 0 & t < 0, \\ t & 0 \leq t < 1, \\ 1 & t \geq 1. \end{cases}$$

Here $\mathbb{P}(\cdot)$ (script P) denotes probability. The shorthand $U \sim \mathcal{U}[0, 1)$ is read “ U is distributed according to the uniform distribution on $[0, 1)$ ”. The additional requirement that $U_1, U_2, \dots$ are *independent and identically distributed (i.i.d.)* means that knowing any subset of the U_j ’s gives no information about the others:

$$\mathbb{P}(U_1 \leq t_1, \dots, U_n \leq t_n) = t_1 \cdot t_2 \cdots t_n, \quad 0 \leq t_i \leq 1.$$

(ii) Uniform random bits. The B_j are i.i.d. taking values in $\{0, 1\}$ with $\mathbb{P}(B_j = 0) = \mathbb{P}(B_j = 1) = \frac{1}{2}$.

2.2 Random Permutations: Shuffling and its Applications I

A **permutation** of the set $\{1, 2, \dots, n\}$ is simply an arrangement of its elements in some order. For example, with $n = 3$ the possible permutations are $(1, 2, 3)$, $(1, 3, 2)$, $(2, 1, 3)$, $(2, 3, 1)$, $(3, 1, 2)$, and $(3, 2, 1)$ — six arrangements in total, because there are $3! = 3 \times 2 \times 1 = 6$ ways to arrange three distinct objects. In general, the number of permutations of $\{1, \dots, n\}$ is $n! = n \times (n - 1) \times \dots \times 2 \times 1$ (read “ n factorial”).

A **random permutation** is one chosen *uniformly at random* from all $n!$ possible arrangements, meaning every arrangement has exactly the same probability $1/n!$ of being selected. Generating random permutations is a fundamental building block of Monte Carlo algorithms. It underlies random shuffling of data, randomised controlled experiments (assigning treatments to subjects), bootstrapping, and permutation-based statistical tests.

The naive approach and why it fails. One might imagine picking elements one at a time from $\{1, \dots, n\}$ uniformly at random, discarding repeats until all n elements have been chosen. This is slow because later draws must avoid an ever-growing list of already-chosen elements — a situation known as the *coupon-collector problem*, where the expected number of draws grows as $n \log n$. The Fisher-Yates shuffle completely avoids this issue by construction.

2.2 Random Permutations: Shuffling and its Applications II

Algorithm 1: Fisher-Yates Shuffle (also called the Knuth Shuffle)

Require: An integer $n \geq 1$ (the number of elements to shuffle); pseudorandom numbers $u_1, u_2, \dots, u_n$ with each $u_i \in [0, 1)$.

Ensure: A uniformly random permutation of $\{1, 2, \dots, n\}$, stored in an array S .

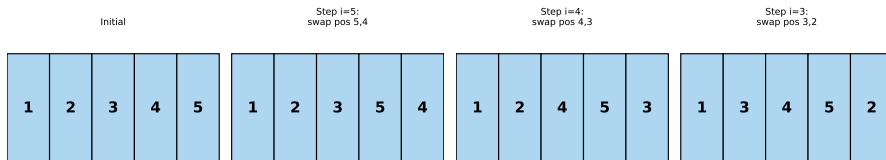
- ① *Initialise:* set $S[i] \leftarrow i$ for $i = 1, \dots, n$ (begin with the identity arrangement: element i sits in position i).
- ② **For** $i = n$ **down to** 1 **do:**
 - ① Compute $k \leftarrow 1 + \lfloor i \cdot u_i \rfloor$. This selects a uniformly random index k from $\{1, 2, \dots, i\}$ (there are exactly i possible values, since $\lfloor i \cdot u_i \rfloor \in \{0, 1, \dots, i - 1\}$).
 - ② *Swap* the elements at positions k and i : exchange $S[k]$ and $S[i]$.
- ③ **Return** S .

2.2 Random Permutations: Shuffling and its Applications III

How to Read This Algorithm

At each step i (counting backwards from n to 1), we pick one of the i elements currently in positions 1 through i uniformly at random and move it into the “final” position i . Once placed in position i , an element is never touched again. After all n steps every element has been placed exactly once, and because at each step i every one of the i remaining candidates is equally likely to be chosen, the overall probability of any particular complete ordering is $\frac{1}{n} \cdot \frac{1}{n-1} \cdots \frac{1}{1} = \frac{1}{n!}$. The algorithm runs in exactly n swap operations, which is optimal (you cannot shuffle n elements with fewer than $n - 1$ operations).

Fisher-Yates Algorithm: Shuffling [1, 2, 3, 4, 5]



2.2 Random Permutations: Shuffling and its Applications IV

The figure above shows a trace of Algorithm 1 for $n = 5$. At each step the chosen element (highlighted) is swapped into its final position. After all five steps the resulting arrangement is guaranteed to be uniformly random.

2.2 Random Permutations: Shuffling and its Applications V

Correctness of Fisher-Yates: a rigorous inductive argument.

We prove by induction on the loop counter i that after the algorithm has processed positions $n, n-1, \dots, i+1$, the elements now occupying positions $i+1, i+2, \dots, n$ form a uniformly random arrangement of the $n-i$ elements assigned to those positions. This invariant guarantees that the final result is a uniformly random permutation of all n elements.

Base case ($i = n$): At the first step, we pick a uniformly random index k from $\{1, \dots, n\}$ and swap $S[k]$ with $S[n]$. Every element of $\{1, \dots, n\}$ is equally likely to end up in position n , so the element in position n is uniformly distributed. The invariant holds.

Inductive step: Assume the elements in positions $i+1, \dots, n$ are already a uniformly random arrangement of their $n-i$ elements. The i elements remaining in positions $1, \dots, i$ are therefore equally likely to be any particular i -element subset of $\{1, \dots, n\}$ not already placed. We pick one of these i remaining elements uniformly at random and move it to position i . The invariant is maintained. After step $i = 1$, every arrangement of all n elements occurs with probability $1/n!$, completing the proof.

2.2 Random Permutations: Shuffling and its Applications VI

Algorithm 2: Perm-Sort — Generating a Random Permutation by Sorting

An alternative approach: assign a uniformly random “key” to each element and then sort by key. The element whose key is smallest goes first, and so on.

- 1 Initialise: $S[i] \leftarrow i$ for $i = 1, \dots, n$.
- 2 Generate $u_1, \dots, u_n$ independently from $\mathcal{U}[0, 1)$.
- 3 Sort S so that $u_{S[1]} \leq u_{S[2]} \leq \dots \leq u_{S[n]}$ (arrange the elements so their assigned keys are in increasing order).
- 4 **Return** S .

Both algorithms produce a uniformly random permutation when given truly uniform input. Fisher-Yates is preferred in practice because it runs in $\mathcal{O}(n)$ time — one swap per step, so exactly n swaps in total. Perm-Sort requires $\mathcal{O}(n \log n)$ time for the sorting step, which is significantly slower for large n . However, Perm-Sort is sometimes easier to parallelise because the sorting step can be distributed across multiple processors.

2.2 Random Permutations: Shuffling and its Applications VII

2.2.2 Shuffling a Deck of Cards: How Many Shuffles Are Enough?

A standard deck of $n = 52$ cards can be identified with the set $\{1, 2, \dots, 52\}$. There are $52! \approx 8.07 \times 10^{67}$ possible orderings — a number so fantastically large that if you shuffled a different arrangement every second from the Big Bang until now, you would have visited only an infinitesimally tiny fraction of all possible decks. In practice, this means a new physical shuffle never repeats a previous deck ordering.

A *card-shuffling scheme* is a Markov chain $(X_k)_{k \geq 0}$ on the symmetric group $\mathcal{S}_{52}$ (the mathematical name for the set of all $52!$ permutations of 52 elements). The chain starts in the initial ordering $X_0 = e$ (the identity permutation) and at each step applies one random shuffle, moving from state X_k to state X_{k+1} . The goal is for the chain's distribution to converge to the *uniform distribution*: every ordering equally likely.

The distance from this ideal uniform distribution after k shuffles is measured by the **total variation distance** d_k (pronounced “dee-sub-k”):

$$d_k = \frac{1}{2} \sum_{\sigma \in \mathcal{S}_n} \left| \mathbb{P}(X_k = \sigma) - \frac{1}{n!} \right|.$$

2.2 Random Permutations: Shuffling and its Applications VIII

Here Σ (Sigma, capital) denotes the sum over all $n!$ permutations σ (sigma) in $\mathcal{S}_n$. The term inside the sum measures how much the probability of each ordering σ differs from the ideal uniform probability $1/n!$. The factor $\frac{1}{2}$ normalises the distance so that $d_k \in [0, 1]$. When $d_k = 1$ the deck is in a completely predictable (deterministic) state; when $d_k = 0$ the deck is perfectly mixed. As more shuffles are applied, $d_k \rightarrow 0$.

How to Read the Total Variation Distance

Think of d_k as a “percentage of remaining non-randomness”. If $d_k = 0.5$, for example, then an adversary who could always choose the most over-represented ordering could win a card game with probability $0.5 + 0.5 \times 0.5 = 0.75$ instead of the fair $1/n!$. Practical applications typically aim for $d_k < 0.01$ before considering the deck “sufficiently shuffled”.

Common shuffling methods and their theoretical properties.

The *Top-To-Random* shuffle: remove the top card and insert it into a uniformly random position among the n available slots in the deck. This is the simplest scheme to analyse mathematically because the

2.2 Random Permutations: Shuffling and its Applications IX

transition probabilities are easy to compute, but it requires about $n \ln n \approx 235$ shuffles to mix a 52-card deck — far too many for practical use.

The *Riffle Shuffle (Gilbert-Shannon-Reeds model)*: cut the deck into two halves according to a binomial distribution $\text{Bin}(n, 1/2)$ (this models the fact that in a real riffle shuffle each of the n cards independently ends up in the left or right half with equal probability), then interleave the two halves by repeatedly dropping a card from whichever pile is larger, with probability proportional to the current pile sizes. This precisely models the riffle shuffle that people perform physically at a card table.

The *Time-Reversed Riffle Shuffle*: the time-reverse of the Riffle Shuffle interleaves two packs by independently inserting each card of one pack into the other at a random position. This variant is mathematically useful because its transition probabilities have a cleaner closed form, simplifying the analysis of mixing times.

2.2 Random Permutations: Shuffling and its Applications X

The Seven-Shuffle Theorem (Bayer and Diaconis, 1992)

For a standard 52-card deck, approximately **7 riffle shuffles** are needed before the total variation distance d_k drops below $\frac{1}{2}$. With 6 shuffles the deck is still statistically far from uniformly random; with 8 it is essentially perfectly mixed. This abrupt transition from “unmixed” to “mixed” over a small range of steps — called the *cut-off phenomenon* — was a landmark result in the theory of Markov chain mixing times and has been featured in popular mathematics writing ever since. The proof uses Fourier analysis on the symmetric group $\mathcal{S}_{52}$ and is one of the most elegant applications of pure mathematics to an everyday activity.

2.3 Pseudorandom Number Generators in Python I

Modern scientific computing in Python uses the `numpy.random` module, which provides a carefully designed family of high-quality pseudorandom number generators through a clean, consistent API (application programming interface). This section describes the recommended generators, explains why they are preferred over older alternatives, and shows how to use them in practice.

The recommended entry point for all new Python code is `numpy.random.default_rng(seed)`, which returns a `Generator` object backed by the **PCG64** algorithm. You should use this in every Monte Carlo simulation unless you have a specific reason to do otherwise.

2.3 Pseudorandom Number Generators in Python II

2.3.1 The NumPy Generator Hierarchy: BitGenerators and Generators

NumPy separates the generation of random bits from the conversion of those bits into useful distributions. A *bit generator* (such as PCG64 or MT19937) produces raw pseudorandom bits as fast as possible. A *generator* (the `Generator` class) wraps a bit generator and provides methods like `.random()`, `.integers()`, and `.permutation()` that convert those raw bits into specific distributions. The four main choices of bit generator are:

- **PCG64** (Permuted Congruential Generator, 64-bit output): This is the current NumPy default. It has a period of 2^{128} (approximately 3.4×10^{38}), passes all tests in the TestU01 BigCrush battery, and supports multiple independent streams via the `.advance(n)` method — an essential feature for parallel simulations. Use this unless you have a specific reason not to.
- **MT19937** (Mersenne Twister): The historically dominant generator, with a period of $2^{19937} - 1$. Its state space consists of 624 integers of 32 bits each (totalling 2.5 kilobytes), making it slow to initialise. It fails several tests in the TestU01 BigCrush battery, cannot generate independent parallel streams, and is not suitable for cryptographic use. It remains the default in Python's built-in `random` module and in legacy NumPy code (`np.random.seed`).
- **Philox**: A counter-based generator based on a reduced-round block cipher. It provides provably independent parallel streams by construction, which makes it ideal for highly parallel simulations.

2.3.1 Python Code: Using NumPy PRNGs

```
import numpy as np
from numpy.random import PCG64, MT19937, Generator

# --- Recommended: PCG64 via default_rng -----
# Create a Generator backed by PCG64, seeded at 42 for reproducibility.
# Changing the seed gives a completely different sequence.
rng = np.random.default_rng(seed=42)

u      = rng.random(10)           # 10 uniform numbers in [0, 1)
y      = rng.integers(0, 10, 10)  # 10 integers in {0, 1, ..., 9}
perm   = rng.permutation(52)      # random permutation of {0,...,51}
print("Uniform:", u[:5])
print("Integers:", y[:5])
print("Permutation (first 8):", perm[:8], "...")

# --- Explicitly choose the bit generator -----
# Both lines below produce a Generator, but backed by different algorithms.
rng_pcg = Generator(PCG64(seed=0))
rng_mt  = Generator(MT19937(seed=0))
print("PCG64: ", rng_pcg.random(5))
print("MT19937:", rng_mt.random(5))
# Note: the two sequences are completely different despite the same seed=0,
# because the underlying algorithms differ.

# --- Multiple independent streams (PCG64 advance) -----
# For parallel simulations, we need streams that are statistically
# independent of each other. We achieve this by "advancing" the
```

2.3.2 Cryptographic PRNGs and RC4 I

For *cryptographic* applications (generating encryption keys, producing nonces for authentication protocols, etc.), passing statistical tests is necessary but not sufficient. A **cryptographic PRNG (CSPRNG)** must satisfy a much stronger requirement: it must be *computationally unpredictable*. This means that even if an adversary observes a very long prefix of the output stream, no efficient algorithm should be able to predict the next bit with probability significantly better than $\frac{1}{2}$ (the chance of a random guess).

Why statistical testing alone is not enough for security. A test battery such as TestU01 checks whether the output *looks* uniform and independent: whether it passes every test a statistician might apply. But it says nothing about whether an adversary could recover the generator's internal state from the output. If the state can be recovered, the adversary can compute every past and future output value, breaking any security guarantee entirely. CSPRNGs close this gap by requiring that state recovery be computationally infeasible under standard mathematical hardness assumptions.

RC4 (Rivest Cipher 4) was designed by Ron Rivest in 1987 and became the workhorse of early internet security because of its extreme simplicity and speed. For many years it was used in SSL/TLS (the protocol protecting web traffic), WEP (wireless network encryption), and PDF file encryption. RC4

2.3.2 Cryptographic PRNGs and RC4 II

operates by maintaining a permutation S of $\{0, 1, \dots, n - 1\}$ (with $n = 256$ in standard use, so the permutation has $256!$ possible states) and generating one output byte per step. It consists of two phases:

- **Key-Scheduling Algorithm (KSA):** Initialises the permutation S by mixing in the secret key K . This transforms the identity permutation into a pseudorandom permutation that depends on the key.
- **Pseudo-Random Generation Algorithm (PRGA):** Streams output bytes by continuously updating and querying the permutation. The permutation evolves with every byte produced, so the internal state is never static.

2.3.2 Cryptographic PRNGs and RC4 III

Algorithm 3: RC4 Key-Scheduling Algorithm (KSA)

Require: Integer n (typically 256 for byte-level RC4); key bytes $K[0], K[1], \dots, K[L - 1]$ where L is the key length (for example, $L = 16$ bytes for a 128-bit key).

Ensure: A pseudorandom permutation $S[0], S[1], \dots, S[n - 1]$ of $\{0, 1, \dots, n - 1\}$ that depends on the key.

- ① *Initialise:* $S[i] \leftarrow i$ for $i = 0, 1, \dots, n - 1$ (start with the identity permutation, where each position i holds the value i).
- ② Set $j \leftarrow 0$ (initialise a running index to zero).
- ③ **For** $i = 0$ **to** $n - 1$ **do:**
 - ① $j \leftarrow (j + S[i] + K[i \bmod L]) \bmod n$
(update j using the current state value $S[i]$ and the key byte $K[i \bmod L]$; the notation $i \bmod L$ means the remainder when i is divided by L , so the key “wraps around” if it is shorter than n).
 - ② **Swap** $S[i]$ and $S[j]$.
- ④ Reset $i \leftarrow 0, j \leftarrow 0$ (prepare for the PRGA phase).

The effect of the KSA is that the key byte $K[i \bmod L]$ influences the swap at every position i , mixing the key throughout the entire permutation. Different keys produce different permutations, so the output stream is key-dependent.

2.3.2 Python Code: RC4 Implementation

```
def rc4_ksa(n, key):  
    """  
    Key-Scheduling Algorithm (KSA).  
    n      : state size (use 256 for standard RC4)  
    key    : list of integers in {0,...,n-1} (the secret key bytes)  
    Returns: permutation S[0..n-1] initialised from the key.  
    """  
    S = list(range(n))    # start with the identity permutation  
    j, L = 0, len(key)  
    for i in range(n):  
        # Mix in one key byte at each step.  
        # i % L wraps around if the key is shorter than n.  
        j = (j + S[i] + key[i % L]) % n  
        S[i], S[j] = S[j], S[i]    # swap  
    return S
```

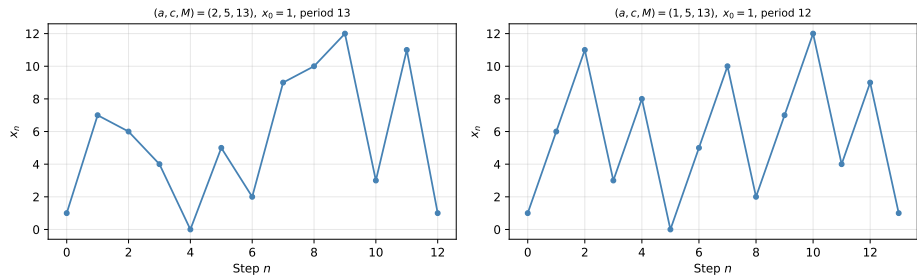
```
def rc4_prga(S, num_bytes):  
    """  
    Pseudo-Random Generation Algorithm (PRGA).  
    S          : permutation from KSA (modified in place)  
    num_bytes  : how many output bytes to generate  
    Returns: list of pseudorandom integers in {0,...,n-1}.  
    """  
    n, i, j = len(S), 0, 0  
    output = []  
    while len(output) < num_bytes:  
        i = (i + 1) % n    # advance counter i  
        j = (j + S[i] + S[S[i]]) % n    # advance pointer j  
        S[i], S[j] = S[j], S[i]    # swap  
        k = S[(S[i] + S[j]) % n]  
        output.append(k % 256)
```

2.4 Simple Generators: Historical Perspective I

Before modern high-quality generators such as PCG64 were developed, practitioners relied on a family of simple, mathematically transparent generators. These are now primarily of historical and educational interest, but studying them in detail is the fastest way to understand *why* modern generators are designed the way they are — and what goes wrong when designers cut corners.

The fundamental challenge: lattice structure. Consider the linear congruential generator (LCG) with the recurrence $x_j = (137 x_{j-1} + 187) \bmod 256$. This generator has a state space of exactly 256 values, so it can at most produce 256 distinct outputs before repeating. If we plot all 256 consecutive pairs (u_i, u_{i+1}) , where $u_i = x_i/256$, we find that the points do *not* fill the unit square uniformly. Instead, every single point lies on one of a small number of parallel diagonal lines. This is the **lattice structure** inherent in all LCGs, and it is immediately visible to the naked eye.

2.4 Simple Generators: Historical Perspective II



The figure above shows consecutive pairs (u_i, u_{i+1}) for a small LCG (left panel) and a somewhat larger one (right panel). In both cases, the parallel-line pattern is unmistakable. Any simulation that uses two or more consecutive numbers from such a generator to form a two-dimensional sample point will produce results that reflect this artificial regularity rather than genuine randomness. For example, a two-dimensional integral estimated by such pairs will systematically miss the regions between the lines, introducing a bias that does not decrease with sample size in the usual way.

2.4 Simple Generators: Historical Perspective III

Key Insight: Lattice Structure Is Invisible in 1D but Fatal in Higher Dimensions

The lattice structure of an LCG is completely invisible when numbers are examined one at a time: the 1D marginal distribution is uniform. It only becomes apparent when consecutive numbers are grouped into pairs, triples, or higher-dimensional vectors. This is why multi-dimensional statistical tests (Section 2.5) are essential: a generator can look perfect in 1D while being catastrophically wrong in 2D.

2.4.1 Linear Congruential Generators (LCG) I

The **linear congruential generator (LCG)** is the simplest useful pseudorandom number generator. It produces a sequence of integers by applying a linear function modulo a large integer, and its appeal is minimal computational cost: one multiplication, one addition, and one modulo operation per step. An enormous body of theoretical work has been done on LCGs, making them the best-understood generators in existence.

The Linear Congruential Recurrence

The LCG produces a sequence $x_0, x_1, x_2, \dots$ according to:

$$x_{n+1} = (a \cdot x_n + c) \bmod M.$$

This formula says: to get the next state x_{n+1} (× subscript n-plus-one), multiply the current state x_n (× subscript n) by the multiplier a , add the increment c , then take the remainder when dividing by the modulus M . The result always lies in $\{0, 1, \dots, M - 1\}$.

2.4.1 Linear Congruential Generators (LCG) II

Notation

- $M > 0$: the **modulus** — the total number of possible states. All outputs x_n lie in $\{0, 1, \dots, M - 1\}$.
- a (where $0 \leq a < M$): the **multiplier** — controls the step size in state space.
- c (where $0 \leq c < M$): the **increment** — an additive offset that prevents the sequence from collapsing to zero.
- x_0 : the **seed** — the user-chosen starting value. Changing the seed produces a different sequence.
- mod (modulo): the remainder after integer division. For example, $17 \bmod 5 = 2$ because $17 = 3 \times 5 + 2$.
- The **output** at step n is $u_n = x_n/M$, which lies in $[0, 1)$.

When $c = 0$, the generator is called a **linear multiplicative congruential generator (LMCG)**.

2.4.1 Linear Congruential Generators (LCG) III

How to Read This Formula

At each step, the sequence “jumps” by a fixed linear rule and wraps around at M . If M is large and the parameters are chosen well, the sequence visits every value in $\{0, \dots, M - 1\}$ exactly once before repeating — achieving the maximum possible period of M . If the parameters are chosen poorly, the sequence may cycle through only a tiny fraction of states, producing highly correlated outputs and a useless generator.

2.4.1 Linear Congruential Generators (LCG) IV

Theorem 2.4.1 — Full-Period Conditions (Knuth)

Let $b = a - 1$. The LCG has period *exactly* M (i.e., visits every state in $\{0, \dots, M - 1\}$ before repeating) if and only if all three conditions hold simultaneously:

- ① $\gcd(c, M) = 1$: the increment c and the modulus M are coprime, meaning they share no common integer factor other than 1.
- ② $b = a - 1$ is divisible by every prime p that divides M . Recall that a prime is a number greater than 1 whose only divisors are 1 and itself (2, 3, 5, 7, 11, ...).
- ③ If 4 divides M , then 4 also divides b . Equivalently: $a \equiv 1 \pmod{4}$, meaning the remainder of a when divided by 4 is equal to 1.

Intuition behind the conditions. Condition 1 ensures the increment does not create a common factor that restricts the sequence to a proper subset of states. For example, if $c = 4$ and $M = 8$, then $\gcd(4, 8) = 4 \neq 1$, and the sequence can only visit even states. Condition 2 ensures that the multiplier “rotates through” all residue classes. Condition 3 is a technical correction needed when M is divisible by a high power of 2, because multiplication behaves differently modulo powers of 2.

2.4.1 Linear Congruential Generators (LCG) V

Worked Example 2.4.2: Classical 32-bit LCG

Choose $M = 2^{31} - 1 = 2,147,483,647$. This particular number is a Mersenne prime (a prime of the form $2^p - 1$), meaning it has no divisors other than 1 and itself. Choose the multiplier $a = 7^5 = 16,807$ and increment $c = 0$ (so this is an LMCG).

Since M is prime, the full-period condition for the LMCG is simply that a is a *primitive root* modulo M , and $16,807 = 7^5$ satisfies this. The period is $M - 1 = 2,147,483,646 \approx 2.1 \times 10^9$. This was among the most widely used generators for 32-bit computers and was proposed by Lewis, Goodman, and Miller.

Other well-known choices include $a = 40,014$ with the same M , and for 36-bit machines:

$M = 2^{35} - 31$, $a = 5^5 = 3,125$.

2.4.1 Linear Congruential Generators (LCG) VI

Worked Example 2.4.3: Small LCG (Resing's Example)

Take $(a, c, M) = (1, 5, 13)$ with seed $x_0 = 1$.

Step 1 — verify full-period conditions: $b = a - 1 = 0$. Since $M = 13$ is prime, its only prime divisor is 13, and $13 \mid 0$ (13 divides 0) trivially. Also $\gcd(5, 13) = 1$ since $5 < 13$ and both are prime. Condition 3 is irrelevant because $4 \nmid 13$. All conditions are satisfied, so the period is $M = 13$.

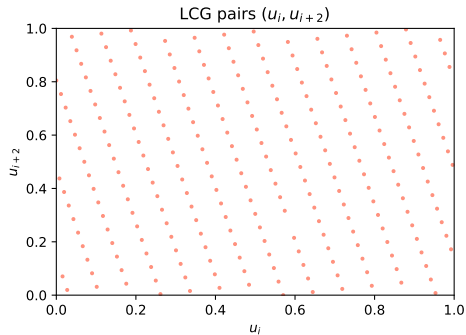
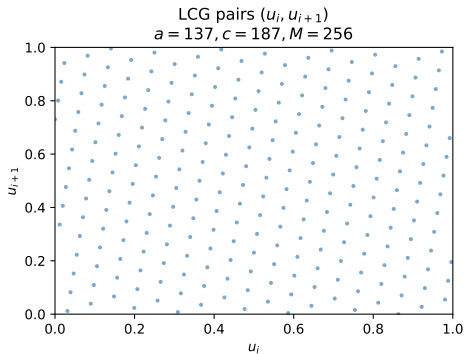
Step 2 — trace the sequence from $x_0 = 1$:

$$1 \rightarrow 6 \rightarrow 11 \rightarrow 3 \rightarrow 8 \rightarrow 0 \rightarrow 5 \rightarrow 10 \rightarrow 2 \rightarrow 7 \rightarrow 12 \rightarrow 4 \rightarrow 9 \rightarrow 1 \rightarrow \dots$$

(Check the first step: $(1 \times 1 + 5) \bmod 13 = 6 \bmod 13 = 6$. Second step: $(1 \times 6 + 5) \bmod 13 = 11$. Third: $(11 + 5) \bmod 13 = 3$. And so on.) The sequence visits all 13 values $\{0, 1, \dots, 12\}$ exactly once before returning to 1 — confirming period 13.

Counter-example: Take $(a, c, M) = (2, 5, 13)$ with $x_0 = 1$. Now $b = a - 1 = 1$. We need every prime dividing $M = 13$ to also divide $b = 1$. But $13 \nmid 1$ (13 does not divide 1), so condition 2 fails. Indeed, the period is only 12, not 13. Moreover, starting from $x_0 = 8$ gives the constant sequence $8 \rightarrow 8 \rightarrow 8 \rightarrow \dots$ with period 1 — a completely degenerate generator.

2.4.1 Linear Congruential Generators (LCG) VII



2.4.1 Linear Congruential Generators (LCG) VIII

The figure shows (u_i, u_{i+1}) pairs for a small-state LCG. The lattice structure — all points lying on a small number of parallel lines rather than filling the square uniformly — is unmistakable. This lattice structure is not an accident: it is an inevitable consequence of the linear arithmetic, described precisely by the following theorem.

Marsaglia's Theorem (1968)

The d -dimensional (d is a positive integer indicating how many consecutive outputs are grouped together) vectors $(u_i, u_{i+1}, \dots, u_{i+d-1})$ produced by *any* LCG with modulus M always lie on a grid of parallel hyperplanes (flat, $(d-1)$ -dimensional subspaces of d -dimensional space). The number of such hyperplanes is at most $M^{1/d}$.

2.4.1 Linear Congruential Generators (LCG) IX

Notation for Marsaglia's Theorem

- d : the dimension (number of consecutive outputs grouped together).
- $M^{1/d}$ (M to the power one-over- d): the d -th root of M . For example, if $M = 10^9$ and $d = 3$, then $M^{1/3} = 10^3 = 1000$ hyperplanes at most.
- A **hyperplane** in d dimensions is the $(d - 1)$ -dimensional analogue of a line (in 2D) or a plane (in 3D). In 2D, it is a line; in 3D, it is a plane.

How to Read Marsaglia's Theorem

The theorem says that no matter how large M is, in higher dimensions the lattice structure *always* appears. As d increases, $M^{1/d}$ decreases: for $M = 2^{31}$ and $d = 6$, the bound is only $(2^{31})^{1/6} \approx 2^{5.2} \approx 37$ hyperplanes. This means a simulation that uses 6-dimensional samples from an LCG might see all its points concentrated on just 37 parallel planes — a catastrophic lack of coverage. This is why LCGs are unsuitable for high-dimensional Monte Carlo.

2.4.1 Linear Congruential Generators (LCG) X

Commercial LCG parameters historically used in widely distributed software.

The table below lists the exact recurrences used in popular software. Each was chosen for implementation simplicity or speed on the hardware of its era. By modern standards, all of them fail the TestU01 BigCrush test battery.

System	Recurrence	Output formula
Java	$x_{i+1} = (25,214,903,917 x_i + 11) \bmod 2^{48}$	$u_i = (2^{27} \lfloor x_{2i}/2^{22} \rfloor + \lfloor x_{2i+1}/2^{21} \rfloor)$
rand48 (UNIX)	same x_i recurrence as Java	$u_i = x_i/2^{48}$
Visual Basic	$x_i = (1,140,671,485 x_{i-1} + 12,820,163) \bmod 2^{24}$	$u_i = x_i/2^{24}$
Excel	$u_i = (0.9821 u_{i-1} + 0.211327) \bmod 1$	direct

The Java formula is particularly instructive: it takes the high bits from two consecutive LCG states and combines them to form a 53-bit double-precision float. This trick improves the 1D output quality but does not fix the 2D lattice structure.

2.4.1 Linear Congruential Generators (LCG) XI

Generalisations of the linear congruential method.

The **generalised linear congruential generator (GLCG)** extends the basic recurrence to use the k (k is a positive integer, the *order* of the recurrence) most recent states:

$$x_n = (a_1 x_{n-1} + a_2 x_{n-2} + \cdots + a_k x_{n-k} + c) \bmod M.$$

2.4.1 Linear Congruential Generators (LCG) XII

Notation

- k : the **order** — how many past states feed into the next state. The ordinary LCG has $k = 1$.
- $a_1, a_2, \dots, a_k$: the **coefficients** — weights on each past state.
- c : the **increment** (same as before).
- M : the **modulus** (same as before).

When M is prime and the *characteristic polynomial* $p(\lambda) = \lambda^k - a_1\lambda^{k-1} - \dots - a_k$ is a **primitive polynomial** over the Galois field $\text{GF}(M)$ (the field with M elements, where arithmetic is modulo M), the period reaches the theoretical maximum of $M^k - 1$. Here λ (lambda) is a placeholder variable. For example, with $M = 2$ and $k = 32$, the period can be $2^{32} - 1 \approx 4.3 \times 10^9$ — far exceeding the period of a simple LCG with the same modulus.

Quadratic and cubic generators replace the linear recurrence with nonlinear recurrences, which break the lattice structure at the cost of slower arithmetic (nonlinear operations are more expensive than addition):

2.4.1 Linear Congruential Generators (LCG) XIII

- *QCG I (Quadratic Congruential Generator, type I)*: $x_{i+1} = x_i^2 \bmod M$, where M is a 512-bit prime. The squaring operation (raising x_i to the power 2) introduces nonlinearity.
- *QCG II (type II)*: $x_{i+1} = 2x_i^2 + 3x_i + 1 \bmod 2^{512}$.
- *CCG (Cubic Congruential Generator)*: $x_{i+1} = 2x_i^3 \bmod 2^{512}$, where x_i^3 means x_i cubed (multiplied by itself twice).

2.4.1 Linear Congruential Generators (LCG) XIV

Fibonacci and Lagged Fibonacci Generators.

The simplest extension of the LCG idea uses *two* past states instead of one, analogous to the Fibonacci sequence (where each number is the sum of the two preceding ones). The standard **lagged Fibonacci generator** is:

$$x_n = x_{n-k} + x_{n-l} \bmod M, \quad n \geq \max(k, l),$$

with recommended parameters $k = 100$, $l = 37$, $M = 2^{30}$.

2.4.1 Linear Congruential Generators (LCG) XV

Notation

- k and l (l): the two **lags** — how far back in the sequence we look for the two inputs. Choosing k and l far apart ensures the two inputs are nearly independent.
- $M = 2^{30}$: a power of 2 is chosen so the modulo operation can be implemented as a fast bitwise AND instead of integer division.
- The sequence requires $\max(k, l) = 100$ initial values (a seed of length 100) before it can start generating outputs.

The period of the lagged Fibonacci generator with these parameters is approximately $(2^{100} - 1) \times 2^{30} \approx 2^{130}$ — very long, with simple arithmetic. However, the 100-value initialisation seed is large and must itself be generated by a good PRNG.

L'Ecuyer Combined Generator. To achieve both a very long period and reduced lattice structure, L'Ecuyer (1988) combined *two* GLCG sequences. The key observation is that if two LCGs have different lattice structures, their combination tends to cancel the worst features of each. The combined output

2.4.1 Linear Congruential Generators (LCG) XVI

has a period approximately equal to the *product* of the two individual periods. The generator is defined by the following four-step procedure:

- ① $x_n = (A_1 x_{n-2} - A_2 x_{n-3}) \bmod M_1$ (first GLCG of order 2)
- ② $y_n = (B_1 y_{n-1} - B_2 y_{n-3}) \bmod M_2$ (second GLCG of order 1 and 3)
- ③ $z_n = (x_n - y_n) \bmod M_1$ (combine by subtraction modulo M_1)
- ④ Output: $u_n = z_n / (M_1 + 1)$ if $z_n > 0$; otherwise $u_n = M_1 / (M_1 + 1)$. (map to $(0, 1)$)

The specific parameter values (chosen by L'Ecuyer to satisfy primitive polynomial conditions on both moduli): $M_1 = 4,294,967,087$, $M_2 = 4,294,944,443$, $A_1 = 1,403,580$, $A_2 = 810,728$, $B_1 = 527,612$, $B_2 = 1,370,589$.

The period of the combined generator is approximately $M_1^2 \cdot M_2 / 2 \approx 2^{191}$, achieved from two components each with period only $\approx 2^{32}$. This is the combination principle at work.

2.4.1 Linear Congruential Generators (LCG) XVII

Key Insight: Combining Imperfect Generators

Combining two generators with different lattice structures typically yields a better generator than either component alone, because the lattice structures are offset and partially cancel each other. This combination principle is the reason L'Ecuyer's generator achieves a period of $\approx 2^{191}$ while each component has period only $\approx 2^{32}$. The same principle underlies all modern composite PRNGs.

2.4.1 Python Code: LCG Implementation and Lattice Visualisation

```
import numpy as np
import matplotlib.pyplot as plt

def lcg(a, c, M, x0, n):
    """Linear Congruential Generator: return n uniform outputs."""
    x, seq = x0, []
    for _ in range(n):
        x = (a * x + c) % M
        seq.append(x / M)
    return seq

# Example 2.4.3: (a,c,M) = (1,5,13), x0=1 => full period 13
u = lcg(a=1, c=5, M=13, x0=1, n=14)
print("LCG (1,5,13), x0=1:", [f"{v:.3f}" for v in u])
# -> [0.462, 0.846, 0.231, 0.615, 0.000, 0.385, 0.769, ...]

# Example 2.4.3 counter-case: period only 12, not 13
u2 = lcg(a=2, c=5, M=13, x0=1, n=14)
print("LCG (2,5,13), x0=1:", [f"{v:.3f}" for v in u2])

# Visualise the lattice structure for a 256-state LCG
u256 = lcg(a=137, c=187, M=256, x0=1, n=256)
plt.figure(figsize=(5, 4))
plt.scatter(u256[:-1], u256[1:], s=4, alpha=0.7)
plt.title("LCG pairs: visible lattice structure")
plt.xlabel(r"$u_i$"); plt.ylabel(r"$u_{i+1}$")
plt.tight_layout(); plt.savefig("lcg_scatter.pdf")
```

2.4.2 Bit Generators: Building Randomness from Binary Operations I

Many pseudorandom number generators operate directly on *bits* (values of 0 or 1) rather than integers. These **bit generators** exploit the fact that a modern computer can perform operations on an entire 64-bit word (a sequence of 64 binary digits stored together) in a single clock cycle. The operations used are XOR (exclusive or) and bit shifts, both of which are extremely fast at the hardware level. Cryptographic generators also inherently operate at the bit level, since encryption and decryption are fundamentally bitwise.

2.4.2 Bit Generators: Building Randomness from Binary Operations II

Fundamental Bit Operations

Exclusive OR (XOR), written $\oplus$ (read “ex-or” or “XOR”): returns 1 if and only if exactly one of its two inputs is 1. For single bits: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

For multi-bit words, XOR is applied element-by-element (position by position). Example with 4-bit words: $1011 \oplus 0110 = 1101$.

Two important properties of XOR:

- ① $x \oplus x = 0$ for any x : XOR-ing a value with itself always gives zero.
- ② $x \oplus 0 = x$ for any x : XOR-ing with zero leaves the value unchanged.

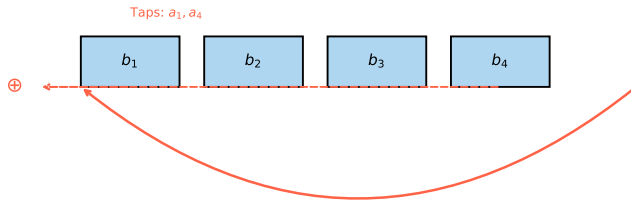
Left shift $\ll a$ (“shift left by a positions”): moves all bits a positions to the left, filling the a vacated positions on the right with zeros. This is equivalent to multiplying by 2^a (2 to the power a) in ordinary arithmetic, but implemented in hardware without actual multiplication. For example, $x \ll 2$ multiplies x by 4.

Right shift $\gg b$ (“shift right by b positions”): moves all bits b positions to the right, discarding the b rightmost bits and filling the b vacated positions on the left with zeros. This is equivalent to integer division by 2^b . For example, $x \gg 3$ divides x by 8 and discards the remainder.

2.4.2 Bit Generators: Building Randomness from Binary Operations III

Linear Feedback Shift Register (LFSR)

$$X_i = (a_1 X_{i-1} \oplus a_2 X_{i-2} \oplus \cdots \oplus a_k X_{i-k})$$



The figure shows the schematic of a Linear Feedback Shift Register (LFSR). A k -bit register (a sequence of k storage cells, each holding one bit) is advanced one step by shifting all bits right by one position and computing a new leftmost bit as the XOR of selected positions (the “taps”).

2.4.2 Bit Generators: Building Randomness from Binary Operations IV

Exclusive OR Generator (XORG)

The XORG is one of the simplest bitwise generators, using a lag of 127 steps:

$$x_i = x_{i-1} \oplus x_{i-127}, \quad i \geq 128.$$

Each new bit x_i (x subscript i) is the XOR of the immediately preceding bit x_{i-1} (one step back) and the bit from 127 steps ago x_{i-127} . The generator requires a 127-bit initialisation (seed) $x_1, x_2, \dots, x_{127}$ before it can produce new bits. The lag of 127 gives the generator memory that prevents the trivially short cycles of a 1-step XOR recurrence.

Worked example (tiny version with lag 3, 4-bit seed $x_0 = 1, x_1 = 0, x_2 = 1$):

$x_3 = x_2 \oplus x_0 = 1 \oplus 1 = 0$. $x_4 = x_3 \oplus x_1 = 0 \oplus 0 = 0$. $x_5 = x_4 \oplus x_2 = 0 \oplus 1 = 1$.

$x_6 = x_5 \oplus x_3 = 1 \oplus 0 = 1$. And so on.

2.4.2 Bit Generators: Building Randomness from Binary Operations V

Linear Feedback Shift Register (LFSR)

An LFSR stores k bits in a shift register and computes the new bit from a linear combination of selected positions (called *taps*):

$$x_i = a_1 x_{i-1} \oplus a_2 x_{i-2} \oplus \cdots \oplus a_k x_{i-k}.$$

- k : the **register length** — the number of bits stored.
- $a_j \in \{0, 1\}$ (a subscript j): the **feedback coefficients** — either 0 (meaning position j does not contribute) or 1 (meaning it does). The positions j where $a_j = 1$ are called the *taps*.
- The all-zeros state must be excluded: once all k bits are 0, the LFSR produces zeros forever.
- When the feedback polynomial $p(z) = 1 + a_1 z + \cdots + a_k z^k$ is a **primitive polynomial** over $\text{GF}(2)$ (the field with just two elements, 0 and 1), the LFSR achieves the maximum period of $2^k - 1$. For example, a 32-tap LFSR with the right primitive polynomial has period $2^{32} - 1 \approx 4.3 \times 10^9$.

Critical weakness: LFSRs are not cryptographically secure. Given any $2k$ consecutive output bits, one can recover the entire feedback polynomial by solving a system of $2k$ linear equations over $\text{GF}(2)$ (the Berlekamp-Massey algorithm). Once the polynomial is known, all future and past bits can be computed. For non-cryptographic simulations, LFSRs can be adequate, but they should never be used where security matters.

2.5 Testing Good Properties of PRNGs I

The quality of a pseudorandom number generator directly determines the reliability of any simulation, integral estimate, or statistical analysis built on top of it. A generator that looks good in one dimension may fail badly in two or more dimensions. Statistical testing is how we gain confidence — before committing to a generator for a long, expensive simulation — that its output is *indistinguishable* from truly random output for all practical purposes.

There are two complementary approaches to assessing PRNG quality.

Theoretical tests examine the internal algebraic structure of the generator. The most important example is the *spectral test*: it analyses the geometry of the lattice formed by the d -dimensional vectors $(u_i, u_{i+1}, \dots, u_{i+d-1})$ and measures the minimum distance between consecutive parallel hyperplanes containing all these points. A large minimum distance (meaning the hyperplanes are densely and evenly packed) indicates good d -dimensional uniformity. Theoretical tests can certify quality without running the generator at all, but they only apply to generators whose structure is mathematically tractable.

Statistical tests treat the PRNG as a black box and apply hypothesis tests to the output sequence. The question is: could this sequence plausibly have been produced by a truly random source?

Major test batteries in current use:

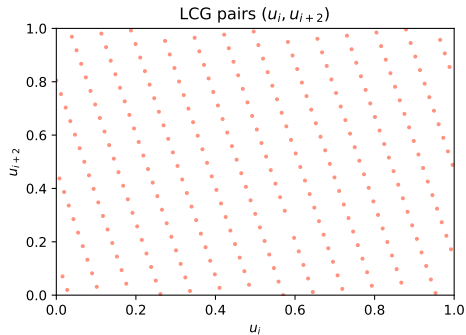
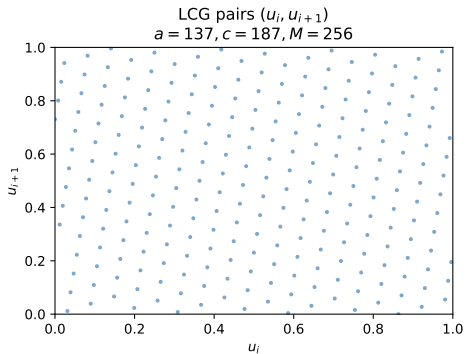
2.5 Testing Good Properties of PRNGs II

Diehard (Marsaglia): 18 tests. Historically important but now superseded. Many generators that pass Diehard fail the more stringent modern tests.

TestU01 (L'Ecuyer and Simard): three nested batteries of increasing severity — SmallCrush (10 tests), Crush (96 tests), BigCrush (106 tests). BigCrush requires approximately 10^{10} (ten billion) random numbers to complete one full run and is the most demanding PRNG test battery in current use. Any generator that fails BigCrush should not be used for serious Monte Carlo work.

NIST SP 800-22: 15 tests designed for cryptographic PRNGs, with emphasis on bit-level properties. These are the tests used by the US government to certify random number generators for cryptographic applications.

2.5 Testing Good Properties of PRNGs III



2.5 Testing Good Properties of PRNGs IV

The figure provides a visual illustration of why testing is essential: pairs (u_i, u_{i+1}) from a simple 256-state LCG lie on a few parallel lines, immediately failing a visual test for 2D uniformity. A good generator's pairs should fill the square densely and uniformly.

Key Insight: A Test Can Only Reject, Never Confirm

No finite set of tests can *prove* that a generator is random — a sequence can pass every test in a battery and still be deterministic with a known structure. Tests can only *reject* generators that are demonstrably non-random. Passing all tests of a respected battery (such as TestU01 BigCrush) gives strong practical confidence but not a mathematical guarantee.

2.5.1 Preliminary Remarks: Notation and Conversions I

Before describing specific tests, we establish notation and address two practical matters: how to correctly convert between output formats, and how to organise a sequence into multi-dimensional vectors for testing.

Standard notation. We use capital letters for random variables (quantities whose values are uncertain until observed) and lowercase letters for specific observed values (realisations):

- $U \sim \mathcal{U}[0, 1)$: U (capital U) is a random variable uniformly distributed on $[0, 1)$. Its realisation (a specific observed value) is denoted u (lowercase u).
- $B \sim \mathcal{U}(\{0, 1\})$: B is a uniform random bit (equally likely to be 0 or 1). Realisation: b .
- $Y \sim \mathcal{U}(\bar{M})$: Y is uniformly distributed on the integers $\{0, 1, \dots, M - 1\}$. Here $\bar{M}$ (M with a bar) is shorthand for this set. Realisation: y .

2.5.1 Preliminary Remarks: Notation and Conversions II

2.5.1.1 Converting Between Output Formats: The Right Way and the Wrong Way

Given a sequence $u_1, u_2, \dots$ of values in $[0, 1)$, convert to integers in $\{0, \dots, M - 1\}$ via the floor function:

$$y_i = \lfloor M u_i \rfloor.$$

Here $\lfloor \cdot \rfloor$ (the floor function, pronounced “floor of”) rounds down to the nearest integer. For example, if $u_i = 0.731$ and $M = 6$, then $y_i = \lfloor 6 \times 0.731 \rfloor = \lfloor 4.386 \rfloor = 4$. Conversely, $u_i = y_i/M$ recovers a $[0, 1)$ value from an integer.

Converting to bits: the correct method. To extract random bits from u_i , first convert u_i to an integer $y_i \in \{0, \dots, 2^k - 1\}$ using $M = 2^k$, then read the individual binary digits of y_i . Each binary digit of y_i is an independent uniform bit.

Why the naive threshold rule is dangerous. The threshold rule $b_i = 1[u_i \geq 0.5]$ (which outputs 1 if $u_i \geq 0.5$ and 0 otherwise) extracts only the *most significant bit* of u_i and discards all remaining bits. Suppose u_i is a 24-bit floating-point number: the threshold rule uses only bit b_1 and ignores $b_2, \dots, b_{24}$. A defective generator could set $b_2 = \dots = b_{24} = 0$ for every output, making every odd-indexed output identically zero. This would pass the threshold test perfectly while failing any proper bit-level test.

2.5.1 Preliminary Remarks: Notation and Conversions III

2.5.1.2 Grouping Numbers into Multi-Dimensional Vectors

Given $n = m \times r$ numbers $u_1, u_2, \dots, u_n$, group them into r vectors of dimension m :

$$\mathbf{u}_1 = (u_1, \dots, u_m), \quad \mathbf{u}_2 = (u_{m+1}, \dots, u_{2m}), \quad \dots, \quad \mathbf{u}_r = (u_{(r-1)m+1}, \dots, u_{rm}).$$

- m : the **dimension** of each vector — how many consecutive outputs form one test point.
- r : the **number of test points** — how many vectors are formed.
- $n = m \times r$: the total number of outputs consumed.
- Each $\mathbf{u}_i$ (bold u subscript i) is a point in the m -dimensional unit hypercube $[0, 1]^m$.

Multi-dimensional grouping is used in virtually all test batteries because independence in m dimensions is much harder to achieve than independence in one dimension. A generator can have a perfectly uniform 1D distribution while being heavily correlated in 2D (as shown by the LCG lattice structure in Figure 2.4.1).

2.5.1 Preliminary Remarks: Notation and Conversions IV

2.5.1.3 The p -Value Concept

Every hypothesis test produces a **p -value** (p-value, where p stands for probability). For a test statistic T (T) with cumulative distribution function (c.d.f.) F (capital F, the function that gives the probability that T is at most a given value) under the null hypothesis $\mathcal{H}_0$ (H-nought, meaning “the sequence is truly random”):

$$p = \mathbb{P}(T \geq T_{\text{obs}}) = 1 - F(T_{\text{obs}}).$$

Here T_{obs} (T subscript “obs”, short for observed) is the value of the test statistic computed from the actual PRNG output, and $\mathbb{P}(\cdot)$ (read “probability of”) gives the probability of an event under $\mathcal{H}_0$. Under $\mathcal{H}_0$ with a continuous test statistic, the p -value is itself uniformly distributed: $p \sim \mathcal{U}[0, 1)$. This means:

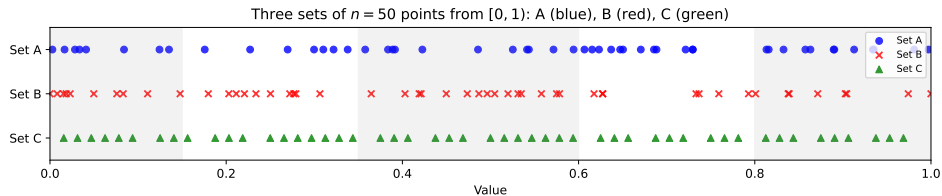
- A *small* p (e.g., $p < 0.01$) is evidence *against* $\mathcal{H}_0$: the observed statistic is unusually large, suggesting the sequence is not random.
- A suspiciously *large* p (e.g., $p > 0.99$) also indicates non-randomness: the sequence is “too uniform” to be natural. This is exactly what happens with quasi-random (low-discrepancy) sequences, which are deliberately designed to be more evenly spread than any truly random sequence could be.

2.5.2 Two Fundamental Goodness-of-Fit Tests I

We introduce the two most fundamental goodness-of-fit tests using three concrete sets of $n = 50$ points from $[0, 1)$, chosen to illustrate three qualitatively different situations:

- **Set A** (blue): generated by PCG64 via NumPy's `default_rng`. This represents a high-quality pseudorandom sequence and serves as the baseline.
- **Set B** (red): a biased nonlinear transform of Set A, given by $u_i^B = (e^{u_i^A - 1} - m)/(M - m)$, where m and M are the minimum and maximum of the transformed values. This set is skewed heavily toward 0 and should be detected as non-uniform.
- **Set C** (green): a quasi-random equispaced sequence, $u_i = i/(n + 1)$ for $i = 1, \dots, n$. This set is “too uniformly distributed” to be random and illustrates the need for second-level testing (Section 2.5.6).

2.5.2 Two Fundamental Goodness-of-Fit Tests II



2.5.2 Two Fundamental Goodness-of-Fit Tests III

2.5.2.1 Kolmogorov-Smirnov (KS) Goodness-of-Fit Test

The KS test compares the **empirical cumulative distribution function** (empirical c.d.f.) to the theoretical c.d.f. of $\mathcal{U}[0, 1)$.

The **empirical c.d.f.** $\hat{F}_n(t)$ is the step function that records the fraction of observed values at or below t :

$$\hat{F}_n(t) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(U_i \leq t).$$

Here $\mathbf{1}(U_i \leq t)$ (the indicator function, pronounced “1 if $U_i \leq t$, and 0 otherwise”) equals 1 when the i -th observation is at most t , and 0 when it exceeds t . So $\hat{F}_n(t)$ simply counts the proportion of the n sample values that do not exceed t .

For a truly uniform $\mathcal{U}[0, 1)$ distribution, the theoretical c.d.f. is $F(t) = t$ (the diagonal line from $(0, 0)$ to $(1, 1)$).

The **KS statistic** D_n measures the largest gap between the empirical and theoretical c.d.f.s:

$$D_n = \sup_{0 \leq t \leq 1} |\hat{F}_n(t) - t|.$$

Here $\sup$ (supremum, from Latin “supremum” meaning the least upper bound) denotes the maximum

2.5.2 Python Code: KS, Chi-Square, and Pairs Tests

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(seed=0)
n = 50
# Set A: PCG64 pseudorandom
A = rng.random(n)
# Set B: biased nonlinear transform (skewed toward 0)
eps = 1e-7
raw = np.exp(A - 1) - 1 - eps
B = (raw - raw.min()) / (raw.max() - raw.min())
# Set C: quasi-random equispaced (Halton-like)
C = np.arange(1, n + 1) / (n + 1)

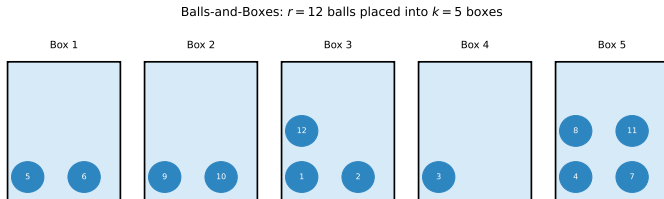
# --- KS test against U[0,1) -----
for name, seq in [("A", A), ("B", B), ("C", C)]:
    stat, p = stats.kstest(seq, 'uniform')
    print(f"KS Set {name}: D={stat:.4f}, p={p:.4f}")

# --- Chi-square test with 5 non-uniform bins -----
bins = [0, 0.15, 0.35, 0.60, 0.80, 1.0]
probs = [0.15, 0.20, 0.25, 0.20, 0.20] # must sum to 1.0
for name, seq in [("A", A), ("B", B), ("C", C)]:
    obs, _ = np.histogram(seq, bins=bins)
    exp = [n * p for p in probs]
    chi2, p_val = stats.chisquare(obs, exp)
    print(f"Chi2 Set {name}: X2={chi2:.3f}, p={p_val:.4f}")
```

2.5.3 Tests Based on Balls-and-Boxes Schemes I

Many classical PRNG tests can be unified within the elegant **balls-and-boxes framework**. The general setup is this: imagine placing r (r) balls into k (k) boxes independently and uniformly at random, so each ball independently chooses a box uniformly from $\{1, \dots, k\}$. Under the null hypothesis $\mathcal{H}_0$ (that the sequence is truly random), the occupancy counts follow a *multinomial distribution*.

To convert a PRNG output sequence into this framework: form r vectors $\mathbf{u}_1, \dots, \mathbf{u}_r$ each of dimension m (m), partition $[0, 1)^m$ into $k = L^m$ equal sub-hypercubes (where L is the number of divisions per axis), and map each vector $\mathbf{u}_i$ to the sub-hypercube it falls into (the “box”). The choice of m , L , and k defines the specific test.



2.5.3 Tests Based on Balls-and-Boxes Schemes II

The Birthday Problem. The classical birthday problem asks: how many students must be in a room before there is a better-than-even chance that at least two share a birthday? Assuming birthdays are uniformly distributed among $k = 365$ days, the probability that all r students have *distinct* birthdays is:

$$\frac{(365)_r}{365^r} = \prod_{j=1}^{r-1} \left(1 - \frac{j}{365}\right).$$

Here $(365)_r = 365 \times 364 \times \cdots \times (365 - r + 1)$ is the **falling factorial** (the product of r integers counting down from 365), and $\prod$ (Pi, uppercase, meaning “product over”) multiplies the $r - 1$ terms together. The probability of at least one collision is $1 - (365)_r/365^r$.

2.5.3 Tests Based on Balls-and-Boxes Schemes III

Worked Computation: Birthday Probability

With $r = 23$ students and $k = 365$ days:

$$p = 1 - \frac{365 \times 364 \times \cdots \times 343}{365^{23}} \approx 1 - e^{-23 \times 22 / (2 \times 365)} = 1 - e^{-\lambda}, \quad \lambda = \frac{23^2}{2 \times 365} \approx 0.725.$$

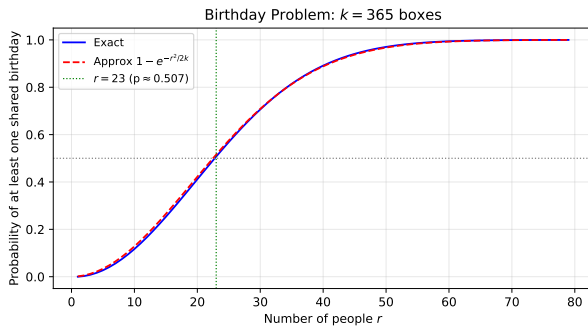
Here $e \approx 2.718$ is Euler's number (the base of the natural logarithm). So $p \approx 1 - e^{-0.725} \approx 0.516$: with 23 students, there is already a 51.6% chance of a shared birthday. With $r = 70$ students, $\lambda \approx 6.7$ and $p \approx 1 - e^{-6.7} \approx 0.999$.

For general k boxes and r balls, the Poisson approximation to the collision probability is:

$$p_{k,r} \approx 1 - \exp\left(-\frac{r^2}{2k}\right). \quad (2.5.12)$$

Here $\exp(\cdot)$ denotes the exponential function (e raised to a power), and $r^2/(2k)$ is the expected number of collisions (denoted λ , lambda).

2.5.3 Tests Based on Balls-and-Boxes Schemes IV



2.5.3 Tests Based on Balls-and-Boxes Schemes V

Collision Test

A *collision* occurs when a ball lands in a box that already contains at least one ball. Let $C_{k,r}$ (C subscript k,r) be the total number of collisions after placing r balls in k boxes. The exact probability of exactly c (c) collisions is:

$$\mathbb{P}(C_{k,r} = c) = \frac{k(k-1) \cdots (k-r+c+1) \cdot S_2(r, r-c)}{k^r}, \quad (2.5.13)$$

where $S_2(r, s)$ (S sub 2, of r and s) is the **Stirling number of the second kind**: the number of ways to partition a set of r labeled objects into exactly s non-empty, unlabeled subsets.

The **Poisson approximation**: when r and k grow large together with $r^2/(2k) \rightarrow \lambda$ (λ (lambda, a fixed positive constant):

$$\mathbb{P}(C_{k,r} = c) \rightarrow \frac{\lambda^c}{c!} e^{-\lambda}. \quad (2.5.14)$$

Here $c!$ (c factorial) is $1 \times 2 \times \cdots \times c$. This says that $C_{k,r}$ follows approximately a Poisson distribution with mean $\lambda = r^2/(2k)$.

2.5.3 Tests Based on Balls-and-Boxes Schemes VI

Expected Number of Collisions

The expected (average) number of collisions is:

$$\mathbb{E}C_{k,r} = r - k \left(1 - \left(1 - \frac{1}{k} \right)^r \right) \approx \frac{r^2}{2k}.$$

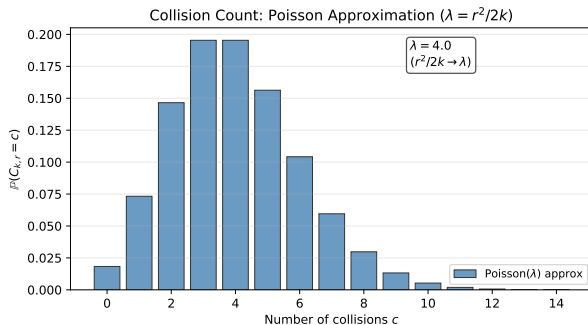
Here $\mathbb{E}$ (E with a double-struck style, read “expected value of”) denotes the average of $C_{k,r}$ over all possible random outcomes. The factor $(1 - 1/k)^r$ is the probability that a given box remains empty after all r balls are thrown.

Worked example: with $r = 2^{14} = 16,384$ balls and $k = 2^{20} \approx 1,048,576$ boxes:

$$\mathbb{E}C_{k,r} \approx \frac{(2^{14})^2}{2 \cdot 2^{20}} = \frac{2^{28}}{2^{21}} = 2^7 = 128.$$

We expect about 128 collisions. A PRNG producing significantly more or fewer collisions than this expected value fails the collision test.

2.5.3 Tests Based on Balls-and-Boxes Schemes VII



2.5.3 Tests Based on Balls-and-Boxes Schemes VIII

Birthday Spacing Test (Marsaglia). This is one of the most stringent tests for LCGs. Given a long sequence of $R \times n$ bits (where R is the number of repetitions and $n = m \times r$ with $m = 25$ and $r = 2^9 = 512$), the test proceeds as follows:

- 1 Split the sequence into R non-overlapping blocks, each of length n .
- 2 In each block, form $r = 512$ values of $m = 25$ bits each (interpreting each 25-bit string as an integer in $\{0, \dots, 2^{25} - 1\}$). These are the “birthdays”, with “year length” $k = 2^{25}$.
- 3 Sort the birthdays and compute the $r - 1$ consecutive spacings (gaps between adjacent sorted values).
- 4 Count K , the number of *equal spacings* (spacings that appear more than once). Under $\mathcal{H}_0$, K approximately follows a Poisson distribution with mean $\lambda = r^3/(4k)$.
- 5 Apply a chi-square test to the R values of K across all blocks.

2.5.3 Tests Based on Balls-and-Boxes Schemes IX

This test fails the MATLAB `rand('state')` LCG with a p -value of 4.12×10^{-100} — a number so astronomically small that the generator is conclusively non-random.

Poker Test. Group $n = 5r$ integers $y_1, \dots, y_n \in \{0, \dots, M - 1\}$ into r groups of 5. For each group, count the number of *distinct values* s among the five (analogous to counting distinct card denominations in a poker hand). The theoretical probability of a 5-tuple having exactly s distinct values is:

$$p_s = \frac{M(M-1) \cdots (M-s+1) \cdot S_2(5, s)}{M^5},$$

where the Stirling numbers for $s = 1, \dots, 5$ are $S_2(5, 1) = 1$, $S_2(5, 2) = 15$, $S_2(5, 3) = 25$, $S_2(5, 4) = 10$, $S_2(5, 5) = 1$. A chi-square test with $k - 1 = 4$ degrees of freedom is then applied to the observed frequencies.

2.5.3 Tests Based on Balls-and-Boxes Schemes X

Key Insight: Balls-and-Boxes Unifies Many Classical Tests

The balls-and-boxes framework shows that the birthday, collision, spacing, and poker tests are all instances of the same combinatorial idea. The birthday spacing test is especially sensitive to LCGs because the lattice structure causes the spacings between sorted values to be non-uniform — certain spacing values are far more common than they should be, and this regularity is detected by counting equal spacings.

2.5.4 Tests Based on Random Walks I

Random walk tests take a different approach: they convert a binary sequence into a one-dimensional random walk and then exploit the rich mathematical theory of random walks to construct tests with precisely known distributions under $\mathcal{H}_0$.

A **random walk** (also called a drunkard's walk) is a sequence of positions on the number line where each step is equally likely to be $+1$ (one step right) or -1 (one step left). If the bits are truly random, the walk is *symmetric*: it has no tendency to drift in either direction. Departures from symmetry indicate biased bit generation.

2.5.4 Tests Based on Random Walks II

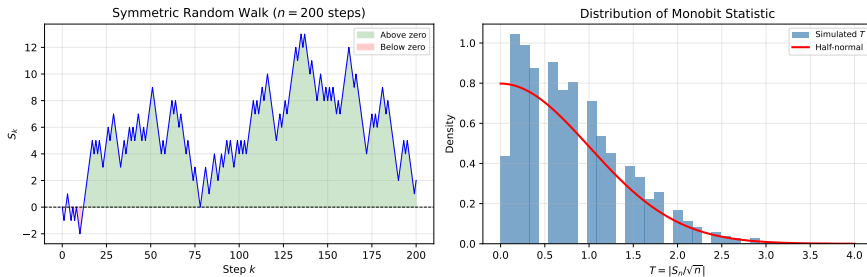
From Bits to a Random Walk

Given bits $b_1, b_2, \dots \in \{0, 1\}$, define the centred increments and partial sums:

$$X_i = 2B_i - 1 \in \{-1, +1\}, \quad S_0 = 0, \quad S_k = \sum_{i=1}^k X_i.$$

Here X_i (X subscript i) is $+1$ when bit $b_i = 1$ (step right) and -1 when $b_i = 0$ (step left). The running sum S_k (S subscript k) gives the position of the walk after k steps. Under $\mathcal{H}_0$, $\mathbb{E}[X_i] = 0$ (the expected step is zero, so there is no drift) and $\text{Var}(X_i) = 1$ (the variance of each step is 1).

2.5.4 Tests Based on Random Walks III



The figure shows a typical symmetric random walk of length $n = 1000$. The walk oscillates without systematic drift. Notice that despite having no systematic bias, the walk can stray far from 0 for extended periods — this is a well-known property of random walks.

2.5.4 Tests Based on Random Walks IV

Frequency (Monobit) Test

The simplest random walk test asks: is the final position S_n consistent with a zero-mean symmetric walk? By the Central Limit Theorem (CLT — the theorem stating that the sum of many independent random variables is approximately normally distributed), $S_n/\sqrt{n}$ converges in distribution to $\mathcal{N}(0, 1)$ (the standard normal distribution, a bell curve centred at 0 with standard deviation 1) as $n \rightarrow \infty$. The test statistic is $T_{\text{obs}} = |S_n|/\sqrt{n}$ (the absolute value of the normalised walk position), and the p -value is:

$$p = \operatorname{erfc}\left(\frac{T_{\text{obs}}}{\sqrt{2}}\right).$$

Here $\operatorname{erfc}(x)$ (complementary error function, pronounced “erfc of x”) is defined as $\operatorname{erfc}(x) = (2/\sqrt{\pi}) \int_x^\infty e^{-t^2} dt$, where π (pi) ≈ 3.14159 . This gives the probability that a standard normal variable has absolute value at least T_{obs} .

The monobit test checks whether the number of 0-bits and 1-bits is approximately equal. If the PRNG produces more 1s than 0s (or vice versa), $|S_n|$ will be large, giving a small p -value and rejecting $\mathcal{H}_0$.

2.5.4 Tests Based on Random Walks V

Frequency Test in Blocks

The global monobit test can miss a generator that is balanced overall but has local imbalances. The block frequency test detects these:

Split $n = m \times r$ bits (m bits per block, r blocks total) into r blocks of length m . For block i , let $S_m^{(i)}$ be the walk position at the end of that block (the sum of ± 1 values within block i). Define the normalised end positions $Z_i = S_m^{(i)} / \sqrt{m}$. For large m , these are approximately i.i.d. $\mathcal{N}(0, 1)$.

The test statistic:

$$T_{\text{obs}} = \frac{1}{m} \sum_{i=1}^r (S_m^{(i)})^2 \approx \chi_r^2 \quad (\text{chi-square with } r \text{ degrees of freedom}).$$

A generator with a long-range trend in bit frequencies (e.g., more 1s in the first half of the sequence and more 0s in the second half) would pass the monobit test but fail the block frequency test, because the individual block sums $S_m^{(i)}$ would not all be near zero.

2.5.4 Tests Based on Random Walks VI

Arcsine Law Based Test

A remarkable and counterintuitive property of the symmetric random walk is the **Arcsine Law** (Feller, 1968). Define L_{2r}^+ as the total number of time steps at which the walk is in positive territory (i.e., the number of indices $k \in \{1, \dots, 2r\}$ with $S_k > 0$). Intuitively, one might expect the walk to spend about half its time above zero and half below. In fact, the opposite is true: the walk is most likely to spend *almost all* its time on one side:

$$\lim_{r \rightarrow \infty} \mathbb{P}\left(\frac{L_{2r}^+}{2r} \leq w\right) = \frac{2}{\pi} \arcsin \sqrt{w}, \quad w \in [0, 1].$$

Here $\arcsin$ (arcsine, the inverse of the sine function) and π (pi) ≈ 3.14159 appear in the formula. The density $\frac{d}{dw} \left[\frac{2}{\pi} \arcsin \sqrt{w} \right] = \frac{1}{\pi \sqrt{w(1-w)}}$ is U-shaped: it is largest near $w = 0$ and $w = 1$. Spending exactly half the time above zero (the “intuitive” outcome) actually has probability zero under this distribution.

This result forms the basis of a test: compute $T_{\text{obs}} = L_{2r}^+ / (2r)$ from the PRNG output and check whether it follows the arcsine distribution. The p -value is: $p \approx 1 - \frac{2}{\pi} \arcsin \sqrt{T_{\text{obs}}}$.

2.5.5 Permutation Tests I

Permutation tests take a complementary approach to random walk tests. Instead of converting bits into walks, they convert blocks of pseudorandom numbers into permutations and then exploit the rich combinatorial theory of permutations to build precise statistical tests.

The key idea is that if we take m (m) uniform numbers $u_1, \dots, u_m$ and sort them to get the ranking (i.e., which is smallest, which is second-smallest, etc.), the ranking should be uniformly distributed over all $m!$ (m factorial) possible orderings. Any deviation from this uniformity reveals structure in the PRNG output.

2.5.5 Permutation Tests II

Derangements (Lemma 2.5.10)

A **derangement** of the set $\{1, 2, \dots, m\}$ is a permutation σ (sigma, lowercase) with *no fixed points*: $\sigma(i) \neq i$ for all $i = 1, \dots, m$. In other words, not a single element stays in its original position after the permutation. A classic example: a derangement of $\{1, 2, 3\}$ is $(2, 3, 1)$ — element 1 moved to position 3, element 2 moved to position 1, element 3 moved to position 2.

The total number of derangements of m elements is:

$$d_m = m! \sum_{k=0}^m \frac{(-1)^k}{k!} = m! \left(1 - 1 + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^m}{m!} \right).$$

Here $(-1)^k$ (negative one to the power k) alternates between $+1$ (when k is even) and -1 (when k is odd). As $m \rightarrow \infty$ (as m grows without bound), the fraction $d_m/m!$ converges to $e^{-1} \approx 0.3679$ (where $e \approx 2.718$ is Euler's number). This means approximately 36.79% of all random permutations of large sets are derangements.

2.5.5 Permutation Tests III

Worked Example: $m = 4$

$$d_4 = 4! \left(1 - 1 + \frac{1}{2} - \frac{1}{6} + \frac{1}{24} \right) = 24 \times \frac{3}{8} = 9.$$

There are 9 derangements of $\{1, 2, 3, 4\}$ out of $4! = 24$ total permutations. The probability of a derangement is $9/24 = 0.375 \approx e^{-1} = 0.3679$ (the approximation improves with larger m).

One such derangement: $(2, 1, 4, 3)$ — element 1 goes to position 2, element 2 to position 1, element 3 to position 4, element 4 to position 3. None of the four elements stays in its original position.

2.5.5 Permutation Tests IV

Derangement Permutation Test

The test uses $n = m \times r$ pseudorandom outputs $u_1, \dots, u_n$, divided into r blocks of m values each.

- 1 Initialise counter $\xi = 0$ (xi, the Greek letter xi, pronounced “ks-eye” or “zai”).
- 2 For each block $i = 1, \dots, r$: use the Fisher-Yates algorithm to generate a permutation σ_i of $\{0, \dots, m-1\}$ from the m uniform inputs $(u_{(i-1)m+1}, \dots, u_{im})$. Increment ξ if σ_i is a derangement.
- 3 Under $\mathcal{H}_0$, the expected number of derangements among r permutations is re^{-1} . The test statistic is:

$$T_{\text{obs}} = \frac{|\xi - re^{-1}|}{\sqrt{re^{-1}(1 - e^{-1})}}, \quad p = \text{erfc}\left(\frac{T_{\text{obs}}}{\sqrt{2}}\right).$$

Notation: ξ (xi): the observed count of derangements. re^{-1} : the expected count of derangements (r times $1/e \approx 0.368$). $\sqrt{re^{-1}(1 - e^{-1})}$: the standard deviation of the count under $\mathcal{H}_0$ (derived from the binomial distribution with r trials and success probability e^{-1}). The denominator normalises the deviation so that T_{obs} follows an approximately standard normal distribution.

2.5.5 Permutation Tests V

Fixed-Point Permutation Test

For each m -tuple, generate permutation σ via Fisher-Yates and count the number of fixed points: $\xi = \#\{j : \sigma(j) = j\}$ (the number of elements that land in their original position).

Under $\mathcal{H}_0$, as $m \rightarrow \infty$, the number of fixed points ξ converges in distribution to a Poisson(1) random variable (a Poisson distribution with mean 1). In a Poisson(1) distribution: $\mathbb{P}(\xi = k) = e^{-1}/k!$ for $k = 0, 1, 2, \dots$

Test procedure: group the r observed fixed-point counts into 11 categories: $A_j = \{j\}$ for $j = 0, 1, \dots, 9$ and $A_{10} = \{10, 11, 12, \dots\}$ (all values ≥ 10). The expected probabilities are $p_j = e^{-1}/j!$ for $j \leq 9$ and $p_{10} = 1 - \sum_{j=0}^9 p_j$. Apply a chi-square test with 10 degrees of freedom.

A PRNG that produces too many or too few fixed points (compared to the Poisson(1) prediction) fails this test. A generator whose permutations are not uniformly distributed will typically have the wrong fixed-point distribution.

2.5.5 Python Code: Permutation Tests

```
import numpy as np
from scipy import stats
from math import factorial, exp

def fisher_yates(m, u):
    """Generate a permutation of {0,...,m-1} from m uniform inputs."""
    S = list(range(m))
    for i in range(m, 0, -1):
        k = int(i * u[i - 1])          # uniform index in {0,...,i-1}
        S[k], S[i - 1] = S[i - 1], S[k] # swap
    return S

def is_derangement(sigma):
    """Return True if sigma has no fixed points."""
    return all(sigma[i] != i for i in range(len(sigma)))

rng = np.random.default_rng(42)
m, r = 10, 2000          # 10-tuples; 2000 repetitions
u = rng.random(m * r)

# ---- Derangement test -----
xi = sum(is_derangement(fisher_yates(m, u[i*m:(i+1)*m]))
         for i in range(r))
e_inv = exp(-1)
T_obs = abs(xi - r * e_inv) / (r * e_inv * (1 - e_inv)**0.5)
p_val = stats.erfc(T_obs / 2**0.5)
print(f"Derangements: {xi}/{r} (expected {r*e_inv:.1f})")
```

2.5.6 p -Values and Second-Level Testing I

A single statistical test applied to a single sequence is often not conclusive. A genuinely good PRNG might occasionally produce a small p -value purely by chance (called a *Type I error* or “false positive”): if we apply 100 tests at the 1% significance level, we expect about 1 false rejection even for a perfect generator. Conversely, a defective generator might happen to pass a particular test for a particular choice of parameters.

Second-level testing addresses both problems by applying the same test to *many* independent sequences and checking that the collection of resulting p -values is itself uniformly distributed over $[0, 1)$.

2.5.6 p -Values and Second-Level Testing II

Why p -Values Are Uniform Under $\mathcal{H}_0$

Suppose the test statistic T has a continuous, strictly increasing cumulative distribution function F under $\mathcal{H}_0$. The p -value is $p = 1 - F(T)$. Then:

$$\mathbb{P}(p \leq t) = \mathbb{P}(1 - F(T) \leq t) = \mathbb{P}(F(T) \geq 1 - t) = \mathbb{P}(T \geq F^{-1}(1 - t)) = 1 - F(F^{-1}(1 - t)) = t.$$

So $p \sim \mathcal{U}[0, 1)$ under $\mathcal{H}_0$. This derivation uses only the fact that F is continuous and strictly increasing, so F^{-1} (F inverse) is well-defined. The conclusion: if the null hypothesis is true, p -values are uniformly distributed. Any systematic departure from uniformity in a large collection of p -values is evidence of non-randomness.

2.5.6 p -Values and Second-Level Testing III

Second-Level Testing Procedure

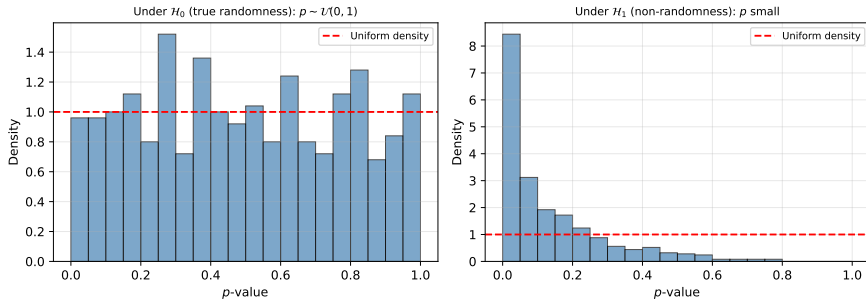
- ① Split a long sequence of total length $R \times n$ into R non-overlapping subsequences, each of length n .
- ② For subsequence i ($i = 1, \dots, R$): compute the test statistic $T^{(i)}$ and first-level p -value $p^{(i)} = 1 - F(T^{(i)})$.
- ③ Test whether $p^{(1)}, p^{(2)}, \dots, p^{(R)}$ are themselves uniformly distributed on $[0, 1)$ by:
 - Ⓐ Partitioning $[0, 1)$ into s equal bins $[(j-1)/s, j/s)$ for $j = 1, \dots, s$.
 - Ⓑ Counting $O_j = |\{i : p^{(i)} \in \text{bin } j\}|$ (the number of first-level p -values falling in bin j).
 - Ⓒ The expected count in each bin is $E_j = R/s$.
 - Ⓓ Computing the chi-square statistic:

$$X_{\text{final}}^2 = \sum_{j=1}^s \frac{(O_j - R/s)^2}{R/s} \approx \chi_{s-1}^2.$$

- ④ The final p -value is $p_{\text{final}} = 1 - F_{\chi_{s-1}^2}(X_{\text{final}}^2)$, where $F_{\chi_{s-1}^2}$ is the c.d.f. of the chi-square distribution with $s - 1$ degrees of freedom.

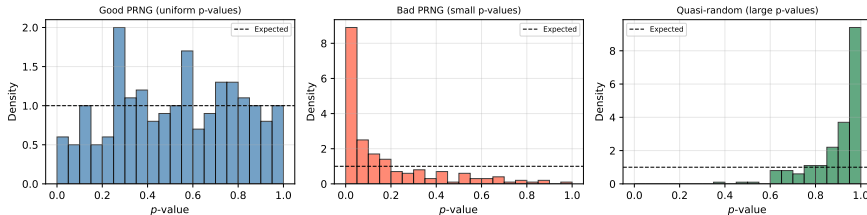
2.5.6 p -Values and Second-Level Testing IV

p -value Distributions



2.5.6 p -Values and Second-Level Testing V

Second-Level Testing: Histograms of $R = 200$ p -values



2.5.6 p -Values and Second-Level Testing VI

Practical recommendations (NIST SP 800-22). The NIST standard recommends using $1000 \leq R \leq 10,000$ subsequences, each of length $2^{20} \leq n \leq 2^{32}$. Use significance level α (alpha) in the range $[0.001, 0.01]$.

The *empirical pass rate* is the fraction of subsequences whose first-level p -value exceeds the threshold α :

$$\hat{B} = \frac{1}{R} \sum_{i=1}^R \mathbf{1}[p^{(i)} > \alpha].$$

Here $\hat{B}$ (B-hat, where the hat denotes “estimated”) should be close to $1 - \alpha$ (the theoretical fraction of sequences that pass). The NIST standard requires $\hat{B}$ to lie within three standard deviations of this expected value:

$$\hat{B} \in \left[1 - \alpha \pm 3\sqrt{\frac{\alpha(1 - \alpha)}{R}} \right].$$

2.5.6 p -Values and Second-Level Testing VII

Example 2.5.12: Second-Level KS Test on Sets A, B, C ($R = 200$)

Apply the KS test to $R = 200$ length- $n = 50$ subsequences:

Set A (PCG64): The 200 first-level p -values spread roughly uniformly across $[0, 1)$. The second-level chi-square test gives a large p -value (near 50%), confirming that the first-level p -values look uniform. Set A **passes**.

Set B (biased): Every subsequence exhibits the same leftward skew, so every first-level p -value is small (near 0). The histogram of p -values is heavily concentrated at the left end. The second-level chi-square statistic is enormous and $p_{\text{final}} \approx 0$. Set B **fails**.

Set C (quasi-random): Every subsequence is “too uniform”, so every first-level KS p -value is large (near 1.0). The histogram is concentrated at the right end. The second-level chi-square statistic is again enormous and $p_{\text{final}} \approx 0$. Set C is **detected as non-random**, even though the first-level KS test did not reject it.

2.5.6 p -Values and Second-Level Testing VIII

Key Insight: Second-Level Testing Is Essential

Second-level testing is strictly more powerful than first-level testing for detecting subtle non-randomness. It is the *only* reliable method to detect quasi-random sequences that fool first-level tests by being “too uniform”. It is also the primary defence against overfitting: a generator whose parameters were tuned to pass one particular first-level test will still fail the second-level test if its p -values cluster near a single value rather than spreading uniformly across $[0, 1)$.

2.5.6 Python Code: Second-Level Testing

```
import numpy as np
from scipy import stats

def second_level_test(sequences, test_fn, s=10):
    """
    Apply test_fn to each sequence to obtain a first-level p-value.
    Then check whether those p-values are U[0,1) using a chi-square
    uniformity test with s equal bins.

    Parameters
    -----
    sequences : list of 1D arrays    (each is one subsequence)
    test_fn    : callable returning a float p-value for one sequence
    s          : int, number of equal bins for the second-level test

    Returns
    -----
    p_vals     : array of first-level p-values (length R)
    chi2_stat   : second-level chi-square statistic
    p_final    : second-level p-value (should be ~uniform if H0 true)
    """
    p_vals      = np.array([test_fn(seq) for seq in sequences])
    obs, _      = np.histogram(p_vals, bins=np.linspace(0, 1, s + 1))
    exp         = [len(p_vals) / s] * s
    chi2_stat, p_final = stats.chisquare(obs, exp)
    return p_vals, chi2_stat, p_final
```

2.6 Bibliographical Notes I

This chapter draws on five decades of PRNG research. The references below provide essential deeper reading for each major topic.

Foundational references on generators:

Knuth [107], *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed., 1997): the definitive mathematical treatment of LCGs. Proves the full-period conditions (Theorem 2.4.1), analyses the spectral test in detail, covers the Fibonacci and Lagged Fibonacci generators, the serial test, and the poker test. The most comprehensive single reference on practical PRNG theory.

L'Ecuyer [118, 122]: designed the combined generators described in Section 2.4.1 and co-developed the TestU01 test suite with Simard. His combined GLCG paper (1988) introduced the principle of using multiple generators with offset lattice structures to achieve very long effective periods. His TestU01 paper (2007) remains the benchmark reference for PRNG testing: BigCrush comprises 106 statistical tests and requires approximately 10^{10} random numbers to complete a single run.

Marsaglia [141, 145]: invented the Diehard test battery (the first systematic battery for testing PRNGs) and the birthday spacing test; his 1968 paper proved the lattice structure result that is now named Marsaglia's theorem. The Diehard battery was widely used through the 1990s and 2000s.

2.6 Bibliographical Notes II

NIST SP 800-22 [187]: the US National Institute of Standards and Technology statistical test suite for cryptographic PRNGs. Contains 15 standardised tests including the monobit test, block frequency test, runs test, and random excursion test. Any generator used in a cryptographic context should pass this suite as a minimum requirement.

2.6 Bibliographical Notes III

Modern generators:

PCG64 (O'Neill [162]): Permuted Congruential Generators (2014). The current NumPy default, accessible via `default_rng`. Uses a 128-bit LCG internal state with a nonlinear output permutation that eliminates the lattice structure while maintaining a period of 2^{128} . Passes all TestU01 BigCrush tests. Supports jumping ahead to arbitrary positions, enabling multiple independent parallel streams.

MT19937 (Matsumoto and Nishimura [146], 1998): Mersenne Twister. Period $2^{19937} - 1$; state of 624 32-bit integers (2.5 KB). Passes Diehard but fails some TestU01 BigCrush tests. Not suitable for cryptographic use. Was the NumPy default before PCG64 and is still the default for Python's built-in `random` module.

AES-CTR and *ChaCha20*: cryptographically secure stream ciphers that have replaced RC4 in modern protocols (RFC 7539 and RFC 8439). Both have formal security proofs and are the recommended choices for any application where the PRNG output must be computationally unpredictable. ChaCha20 is particularly attractive because it requires no special hardware support and is resistant to timing side-channel attacks.

Shuffling and card mixing:

2.6 Bibliographical Notes IV

Bayer and Diaconis [14]: proved the “7 shuffle theorem” for the riffle shuffle on 52 cards using Fourier analysis on the symmetric group $\mathcal{S}_{52}$. This 1992 paper introduced the “cut-off phenomenon” for card shuffling.

Aldous and Diaconis [2]: Markov chain coupling methods for mixing-time analysis; the theoretical foundation for Section 2.2.2.

Statistical tests and their mathematical foundations:

Weiss [210]: central limit theorem for the empty-boxes count (Proposition 2.5.9).

Holst [83] and *L'Ecuyer et al.* [122]: known results on the chi-square distribution of the serial test statistic.

Tsang et al. [205]: analysis of the random excursion test including the theoretical distribution of excursion visit counts.

L'Ecuyer and Simard [169, 170]: design of reliable statistical tests using Berry-Esseen bounds to control normal-approximation errors; explains why TestU01 achieves precisely controlled Type I error rates.

Feller [55], *An Introduction to Probability Theory and Its Applications, Volume I*: classical reference for the arcsine law and other random walk theorems used in Section 2.5.4.

2.7 Selected Exercises I

The following exercises reinforce the theoretical foundations of this chapter. Theoretical exercises are labelled “T”; computational lab exercises are labelled “L”.

2.T.1 (Markov and Chebyshev Inequalities). Two fundamental inequalities bound the probability of large deviations.

For any *non-negative* random variable X (one that is always ≥ 0) and any constant $a > 0$, the **Markov inequality** states:

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}X}{a}.$$

Here $\mathbb{E}X$ (expected value of X) is the average of X over all possible outcomes, and $\mathbb{P}(X \geq a)$ is the probability that X is at least as large as the threshold a . The inequality says: the probability of X being very large is bounded by the ratio of its average to that large value.

For a real-valued X with finite variance $\text{Var}X = \mathbb{E}[(X - \mathbb{E}X)^2]$ and any constant $b > 0$, the **Chebyshev inequality** states:

$$\mathbb{P}(|X - \mathbb{E}X| \geq b) \leq \frac{\text{Var}X}{b^2}.$$

2.7 Selected Exercises II

Here $|X - \mathbb{E}X|$ (the absolute value of X minus its expected value) measures how far X is from its mean. The Chebyshev inequality follows from the Markov inequality applied to the non-negative variable $(X - \mathbb{E}X)^2$: set $a = b^2$ in Markov to get $\mathbb{P}((X - \mathbb{E}X)^2 \geq b^2) \leq \text{Var}X/b^2$.

2.T.4 (Integer Conversion). Let $U_1, U_2, \dots$ be i.i.d. (independent and identically distributed) $\mathcal{U}[0, 1)$ and $M \geq 2$ an integer. Show that the sequence $Y_i = \lfloor M U_i \rfloor$ (floor of M times U_i) consists of i.i.d. uniformly distributed integers on $\{0, 1, \dots, M-1\}$.

Hint: compute $\mathbb{P}(Y_i = k) = \mathbb{P}(k/M \leq U_i < (k+1)/M)$ using the definition of the uniform distribution. This probability should equal $1/M$ for every $k \in \{0, \dots, M-1\}$, confirming that Y_i is uniform on $\bar{M}$.

2.7 Selected Exercises III

2.T.8 (Correctness of Fisher-Yates). Verify that Algorithm 1 (Fisher-Yates shuffle) produces a uniformly random permutation of $\{1, \dots, n\}$.

Hint: proceed by induction on the swap steps. After step i (processing position i), show that the element now in position i is uniformly distributed over the i elements not yet placed in higher positions. A permutation is uniform if and only if each element is equally likely to end up in each position, and the Fisher-Yates induction establishes exactly this.

2.T.17 (Serial Test Mean). Let O_i ($i = 1, \dots, k$) be the number of random vectors $\mathbf{u}_j$ falling into box i in the serial test, where r vectors of dimension m are thrown into $k = L^m$ boxes. Show that under $\mathcal{H}_0$: $\mathbb{E}[X_k^2] = k - 1$, where X_k^2 is the chi-square statistic.

Hint: expand $X_k^2 = \sum_{i=1}^k (O_i - r/k)^2 / (r/k)$, compute $\mathbb{E}[O_i] = r/k$ and $\text{Var}(O_i) = r(1 - 1/k)/k$ from the multinomial distribution, and sum across the k bins.

2.T.19 (Riffle Shuffle and Time Reversal). Let $p_{\text{RS}}(\sigma, \sigma')$ denote the one-step transition probability of the Riffle Shuffle (from permutation σ to σ') and $p_{\text{TR-RS}}(\sigma, \sigma')$ the time-reversed Riffle Shuffle. Show that $p_{\text{RS}}(\sigma, \sigma') = p_{\text{TR-RS}}(\sigma', \sigma)$ for any $\sigma, \sigma' \in \mathcal{S}_n$. This means the Riffle Shuffle and its time-reversed version are related by swapping the source and target permutations — a symmetry that simplifies the analysis of mixing times.

2.7 Selected Exercises IV

Lab Exercises:

2.L.1 (LCG Period Verification). Implement the LCG with $M = 2^{11} - 1 = 2047$, $a = 5$, $c = 1$.

- (a) Generate 10^4 output numbers and plot (i, u_i) versus i (index on the x -axis, output on the y -axis). Observe the periodic pattern.
- (b) Find the period computationally by tracing the state sequence until a state repeats.
- (c) Verify Theorem 2.4.1 by checking all three conditions for $(a, c, M) = (5, 1, 2047)$.

2.L.2 (Broken Fisher-Yates). Compare the correct Fisher-Yates algorithm (Algorithm 1) to a “broken” version in which line 3 is replaced by $k = \text{random}(1, \dots, n)$ (drawing the swap index uniformly from the *full* range $\{1, \dots, n\}$ rather than $\{1, \dots, i\}$).

- (a) Generate 10^4 permutations using each version.
- (b) Apply the fixed-point permutation test (Section 2.5.5) to both.
- (c) Explain why the broken version fails: enumerate the $3! = 6$ permutations of $\{1, 2, 3\}$ reachable by the broken version and check that some are more likely than others.

2.7 Selected Exercises V

2.L.3 (Autocorrelation Comparison). Generate $n = 10^5$ numbers from each of MT19937 and the LMCg with $(M, a) = (2^{31} - 1, 16,807)$. Compute the Pearson autocorrelation between $(y_1, \dots, y_{n-1})$ and $(y_2, \dots, y_n)$. Which generator shows less autocorrelation? Compute the autocorrelation for lags 1, 2, 5, and 10 and compare the results.

2.L.5 (Quasi-Random Detection). Let $u_i = (i \cdot \phi) \bmod 1$ for $i = 1, \dots, n = 10^6$, where $\phi = (\sqrt{5} - 1)/2 \approx 0.6180$ is the golden ratio (this is the Kronecker low-discrepancy sequence).

- (a) Apply the KS test and report the p -value.
- (b) Apply the chi-square test with 10 uniform bins and report the p -value.
- (c) Apply the frequency of pairs test ($L = 10$, $r = 500,000$ pairs) and report the p -value.
- (d) Apply second-level KS testing with $R = 1000$ subsequences of length $n/R = 1000$ each. Report the second-level p -value.
- (e) Observe that first-level tests pass (or produce a suspiciously high p -value) while second-level testing conclusively detects the quasi-randomness.

Chapter 2: Summary

Generators:

- A PRNG is a 5-tuple (S, s_0, f, V, g) ; the period is at most $|S|$.
- Three equivalent sequence types: $\mathcal{U}[0, 1)$, random bits, and uniform integers on $\{0, \dots, M - 1\}$.
- Fisher-Yates shuffle produces a uniform random permutation in $\mathcal{O}(n)$ time (optimal).
- The seven-shuffle theorem: 7 riffle shuffles suffice for 52 cards.
- Python: `default_rng/PCG64` (period 2^{128}) and `MT19937` (period $2^{19937} - 1$).
- RC4: KSA + PRGA permutation cipher; deprecated in 2015 (RFC 7465) due to key-correlated biases.
- LCG full-period conditions: $\gcd(c, M) = 1$; every prime of M divides $a - 1$; and

Statistical Testing:

- KS test: $D_n = \sup |\hat{F}_n(t) - t|$; detects 1D non-uniformity.
- Chi-square: $X^2 = \sum (O_i - E_i)^2 / E_i$ with $E_i \geq 5$ per bin.
- Serial (pairs) test: detects 2D correlations invisible to 1D tests.
- Balls-and-boxes: birthday, collision, birthday spacing, poker — all unified by multinomial occupancy theory.
- Random walk: monobit (global bit balance), block frequency (local balance), arcsine law (time above zero), random excursion (visit counts).
- Permutation: derangement count and fixed-point distribution, both with known exact theoretical distributions